

Universidad Torcuato Di Tella

Métodos Analíticos y Programación

Prof. Hernán Czemerinski

Clase 11: Archivos JSON

Repaso: Clases, objetos y archivos CSV

Los **archivos CSV** son muy adecuados para describir ciertos tipos de **objetos**.

Ejemplo:

```
class Calzado:
    def __init__(self, mar, mod, pre):
        self.marca = mar
        self.modelo = mod
        self.precio = pre
    ...
```

```
Marca,Modelo,Precio
Nike,Air Max,60000.00
Adidas,Superstar,40000.00
Puma,Suede Classic,35000.00
Reebok,Club C,45000.00
Converse,Chuck Taylor All Star,30000.00
Vans,Old Skool,37500.00
```

Cada línea del archivos CSV describe las propiedades de un Calzado.

Repaso: Clases, objetos y archivos CSV

Los **archivos CSV** son muy adecuados para describir ciertos tipos de **objetos**.

Ejemplo:

```
filename = 'calzados.csv'
f = open(filename)

calzados = []
dict_reader = csv.DictReader(f)
for d in dict_reader:
    mar = d['Marca']
    mod = d['Modelo']
    pre = float(d['Precio'])
    c = Calzado(mar,mod,pre)
    calzados.append(p)
...
```

```
Marca,Modelo,Precio
Nike,Air Max,60000.00
Adidas,Superstar,40000.00
Puma,Suede Classic,35000.00
...
```

Cada línea del archivos CSV describe las propiedades de un Calzado.

Repaso: Clases, objetos y archivos CSV

¿Como guardamos datos con **estructuras** más **complejas**?

```
class CarritoDeCompras:
    def __init__(self):
        self.contenido = []

    # Agrega un calzado al carrito
    def agregar(self, calzado):
        self.contenido.append(calzado)

    ...
```

Los CSV tienen varias limitaciones para representar datos con estructuras complejas:

- ▶ asumen una estructura tabular fija, donde cada fila representa un registro y cada columna representa un campo;
- ▶ falta de soporte para anidamiento: no admiten la representación de estructuras anidadas o jerárquicas;
- ▶ no almacenan información sobre los tipos de datos de los campos (todos los valores se tratan como strings).

Archivos JSON

```
{
  "carrito1": [
    {
      "marca": "Nike",
      "modelo": "Air Max 90",
      "precio": 60000.00
    },
    {
      "marca": "Adidas",
      "modelo": "Superstar",
      "precio": 40000.00
    }
  ],
  "carrito2": [
    {
      "marca": "Puma",
      "modelo": "Suede Classic",
      "precio": 35000.00
    }
  ]
}
```

Los archivos JSON:

- ▶ permiten representar estructuras de datos complejas;
- ▶ almacenan información de los tipos de los datos.

Archivos JSON

JSON (*JavaScript Object Notation*) es un formato estándar para almacenar y representar datos.

Ejemplo:

```
{
  "tamano": "mediana",
  "precio": 1499.99,
  "adicionales": ["champinones", "pepperoni", "albahaca"],
  "queso_extra": false,
  "delivery": true,
  "cliente": {
    "nombre": "Paula",
    "telefono": "1133876651",
    "correo": "pau76@em.com"
  }
}
```

- ▶ Un objeto JSON una secuencia de pares clave-valor rodeados por un par de llaves {}.
- ▶ Cada clave se asocia a un valor con este formato: <clave>:<valor>
- ▶ Los pares clave-valor deben estar separados por una coma. Sólo el último par no debe estar seguido de una coma.

Archivos JSON

El formato **JSON** tienen reglas específicas que determinan los **tipos de datos** que son válidos para las claves y para los valores.

Ejemplo:

```
{
  "tamano": "mediana",
  "precio": 1499.99,
  "adicionales": ["champinones", "pepperoni", "albahaca"],
  "queso_extra": false,
  "delivery": true,
  "cliente": {
    "nombre": "Paula",
    "telefono": "1133876651",
    "correo": "pau76@em.com"
  }
}
```

- ▶ Las **claves** deben ser strings.¹
- ▶ Un **valor** puede ser String, Número, Array, Booleano, Objeto JSON.

¹ En JSON los strings se escriben entre comillas dobles.

Archivos JSON: lectura

La función `json.load(file)` carga en un diccionario el contenido del archivo `file`. Este proceso se llama **deserialización**.

```
import json

# Se abre el archivo JSON
f = open('pedido.json')

pedido = json.load(f)      # devuelve un diccionario con el contenido del objeto JSON
print(pedido['tamano'])    # 'mediana'
print(pedido['precio'])    # 1499.99
print(pedido['adicionales']) # ['champinones', 'pepperoni', 'albahaca']
print(pedido['queso_extra']) # False
print(pedido['delivery'])  # True
print(pedido['cliente'])   # {'nombre': 'Paula', 'telefono': '1133876651', 'mail': 'pau76@em.com'}
```

Los tipos de datos de JSON se convierten a tipos de datos en Python:

JSON	Python
String	str
Número (entero)	int
Número (real)	float
Array	list
Booleano (true/false)	bool (True/False)
Objeto JSON	dict
null	None

Archivos JSON: lectura

Ejercicio. (1) Considere archivos JSON para almacenar información de estudiantes universitarios. A continuación se muestra un ejemplo de la estructura de estos archivos.

```
{
  "legajo": "134A23",
  "nombre": "Ana Pérez",
  "edad": 20,
  "carrera": "Ciencia de Datos",
  "materias": [
    {"nombre": "Programación I", "nota": 8},
    {"nombre": "Álgebra", "nota": 6}
  ]
}
```

- (a) Escribir una función en Python `mostrar_info(nom_archivo)` que, dado el nombre de un archivo JSON, muestre por consola el diccionario que resulta de su deserialización.
- (b) Escribir una función en Python `promedio(nom_archivo)` que, dado el nombre de un archivo JSON, calcule el promedio de las notas obtenidas por un estudiante.

Archivos JSON: escritura

```
import json

# Se crean diccionarios con los datos
cliente_json = dict()
cliente_json['nombre'] = 'Manuel'
cliente_json['telefono'] = '1155955953'
cliente_json['mail'] = 'manu14@em.com'

pedido_json = dict()
pedido_json['tamaño'] = 'chica'
pedido_json['precio'] = 1299.99
pedido_json['adicionales'] = ['jamon', 'morron']
pedido_json['queso_extra'] = True
pedido_json['delivery'] = False
pedido_json['cliente'] = cliente_json

f = open('nuevo-pedido.json', 'w') # Se abre el archivo destino en modo escritura
json.dump(pedido_json, f, indent=4) # Se escriben los datos como objeto JSON
f.close() # Se cierra el archivo
```

- ▶ La función `json.dump(dict,file,indent=n)` escribe un objeto JSON en el archivo `file`, a partir de los datos contenidos en `dict`.
- ▶ El parámetro (opcional) `indent` se usa para dar formato al nuevo archivo.

Archivos JSON: escritura

```
{
  "tamano": "chica",
  "precio": 1299.99,
  "adicionales": ["jamón", "morron"],
  "queso_extra": true,
  "delivery": false,
  "cliente": {
    "nombre": "Manuel",
    "telefono": "1155955953",
    "correo": "manu14@em.com"
  }
}
```

- ▶ Al generar el archivo, los tipos de datos de los valores en Python se transforman a tipos de datos JSON (ej: True pasa a ser true).
- ▶ El proceso de generar un objeto JSON a partir de un diccionario Python se llama [serialización](#).

Archivos JSON: escritura

Ejercicio. (2) Considerar la clase Calzado que se describe a continuación:

- ▶ `c = Calzado(mar, mod, pre)`: Crea un nuevo calzado con la marca, el modelo y el precio especificados.
- ▶ `c.marca`: marca del calzado `c`.
- ▶ `c.modelo`: modelo del calzado `c`.
- ▶ `c.precio`: precio del calzado `c`.

Implementar la función `serializar_calzado(cal, nom_archivo)`, que recibe como parámetros una instancia de la clase Calzado y el nombre de un archivo JSON, y debe escribir en dicho archivo la serialización del objeto `cal`.

El siguiente ejemplo ilustra el uso de la función y el contenido que debe generarse en el archivo:

```
from calzado import Calzado

def serializar_calzado(cal, nom_archivo):
    ...

c = Calzado('Puma', 'Suede', 33699.00)
serializar_calzado(c, 'puma_suede.json')
```

```
{
  "marca": "Puma",
  "modelo": "Suede",
  "precio": 33699.00
}
```

Repaso de la clase de hoy

Documentación de referencia: [Biblioteca JSON](#)

- ▶ Archivos JSON
 - ▶ JSON vs. CSV para tipos de datos complejos
 - ▶ Lectura (deserialización) y Escritura (serialización)
 - ▶ Tipos de datos JSON

Con lo visto, ya pueden completar la sección 9 de la guía de ejercicios.